

UNITÉ D'ENSEIGNEMENT: LES FONCTIONS ET LES PROCÉDURES

Pré requis :

- Définition algorithme
- Parties d'un algorithme
- Notion de variable
- Utilisation des structures algorithmiques
- Exécution d'un algorithme simple

Compétences attendues :

- Déclarer une fonction ou procédure
- Identifier le mode de passage d'un paramètre
- Distinguer une variable locale d'une variable globale
- Identifier une fonction ou procédure
- Exécuter un algorithme intégrant une fonction ou une procédure

Situation problème : on désire écrire un algorithme qui va déterminer pour deux nombres entiers s'ils sont positifs ou négatifs, puis afficher la plus petite valeur entre les deux et calculer la somme et la moyenne des deux nombres.

Comment procéder pour écrire cet algorithme ?

- Décomposer le problème en sous problèmes et trouver des solutions pour chacun
- Ecrire les sous algorithmes correspondants
- Réunir le tout dans un algorithme.

INTRODUCTION

Lors de l'écriture des algorithmes importants, il peut arriver qu'une suite d'actions revienne plusieurs fois ou encore on a des suites d'actions qui effectuent des traitements spécifiques différents ce qui entraîne l'obtention de longs algorithmes. Il devient difficile ainsi d'avoir une vision globale de son fonctionnement et de détecter d'éventuelles erreurs. On peut alors définir ces suites d'actions sous forme de sous algorithmes (bloc faisant parti d'un algorithme). Il en existe deux types : les fonctions et les procédures.

I. DÉFINITION ET AVANTAGES

Une fonction est un sous algorithme qui effectue des traitements sur des données en entrée et renvoie un résultat unique à l’algorithme. La fonction n’affiche jamais de résultat à l’écran.

Une procédure est un sous algorithme qui effectue les traitements sur des données en entrée et affiche le ou les résultats obtenus.

Comme avantages des sous algorithmes on a :

- L’amélioration de la lisibilité des algorithmes
- La facilitation de la correction des erreurs
- L’optimisation du nombre d’instructions dans un algorithme

II. DÉCLARATION D’UNE FONCTION /PROCÉDURE

Pour pouvoir utiliser une fonction ou une procédure il faut au préalable la déclarer. On identifie une fonction grâce au mot clé Fonction et une procédure grâce au mot clé Procédure.

Les procédures tout comme les fonctions ont trois parties qui sont :

- L’entête : on y retrouve le mot clé Fonction ou Procédure suivi du nom de la fonction ou de la procédure et ensuite les différents paramètres
- La partie déclaration : on y déclare les variables locales et les constantes
- Le corps : on y trouve les différentes instructions du sous algorithme

1. Déclaration d’une fonction

ete

{ **Fonction** *nom_fonction* (Nom_parametre 1: type ;... ; Nom_parametre n : type) :**type_fonction**

blelocale

{ **Var** // on déclare les variables et constantes

Nom_variable : type

...

s de la

tion

{ **Début**

Instruction1 ;

Instruction 2 ;

....

Instruction n ;

Retourner résultat ;

Fin

Remarque :

- **Type_fonction** correspond au type du résultat renvoyé par la fonction
- **Retourner résultat** est l’instruction qui permet de récupérer le résultat de la fonction

Exemple : Déclaration d’une fonction qui permet de calculer la somme de deux nombres entiers.

Fonction *Somme* (Nbre1 : Entier ; Nbre2 : Entier) : **Entier** ;

Var Somme : Entier

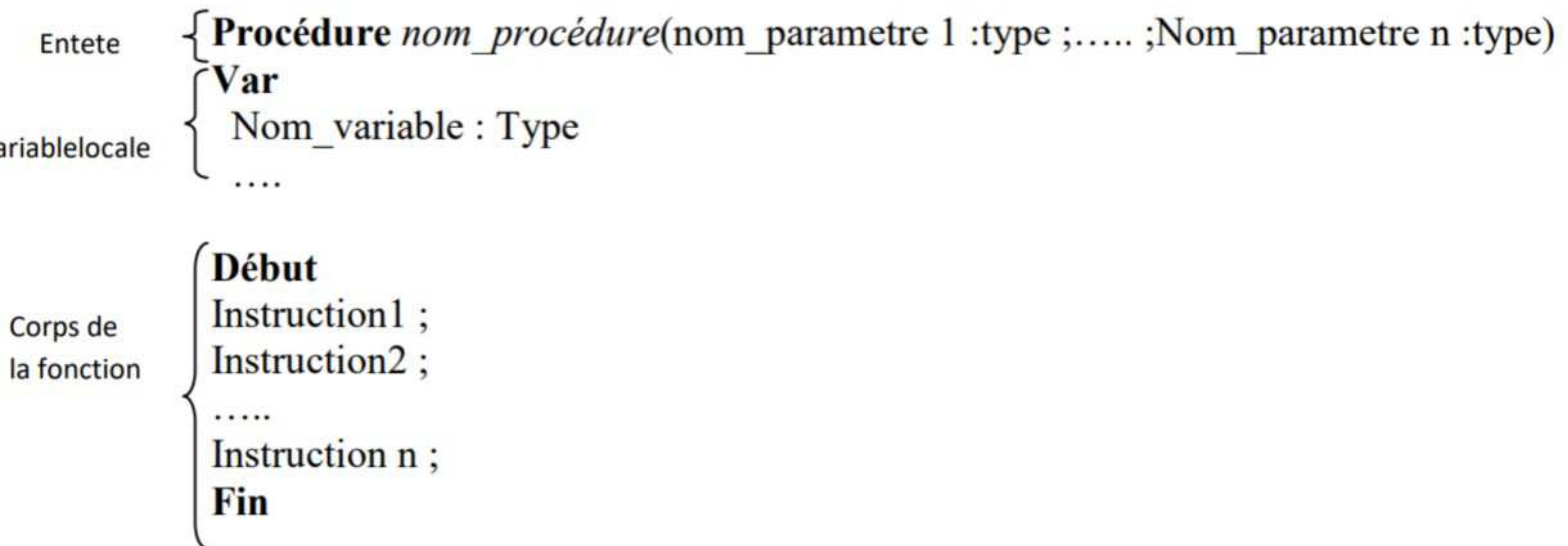
Début

Somme= Nbre1+Nbre2 ;

Retourner Somme ;

Fin

2. Déclaration d'une procédure



Exemple : Déclaration d'une procédure qui permet de calculer la somme de deux nombres entiers.

```
Procédure Somme (Nbre1 : Entier ; Nbre2 : Entier) ;  
  Var Somme : Entier  
  Début  
    Somme= Nbre1+Nbre2 ;  
    Ecrire (" la somme est", Somme) ;  
  Fin
```

Remarque :

- La déclaration d'un sous algorithme ne suffit pas pour qu'elle soit utilisée, il faut l'appeler dans l'algorithme pour qu'elle soit effectivement utilisée.
- L'appel d'un sous algorithme utilise la syntaxe suivante : Nom_sous_algorithme (liste des paramètres)

Exemple :

Algorithme CalculSomme

```
Procédure Somme (Nbre1 : Entier ; Nbre2 : Entier) ;  
  Var Somme : Entier  
  Début  
    Somme= Nbre1+Nbre2 ;  
    Ecrire (" la somme est", Somme) ;  
  Fin  
Var N1, N2 : Entier ;  
Début  
  Ecrire ("Entrez un premier entier") ;  
  Lire(N1) ;  
  Ecrire ("Entrez un deuxième entier") ;  
  Lire(N2) ;  
  Somme (N1 ;N2) ; // appel de la procédure Somme  
FinAlgo
```

III. Portée des variables (Variable locale et variable globale)

L'endroit où est déclarée une variable est très important car il détermine dans quels sous programmes elle va pouvoir être utilisée.

Une variable locale est une variable qui est déclarée dans un sous algorithme et n'est visible que dans ce sous algorithme

Une variable globale est une variable qui est déclarée au niveau de la partie déclaration de l'algorithme et est visible dans tout l'algorithme et dans ses différents sous algorithmes.

Exemple :

Algorithme CalculSomme

Procédure *Somme* (Nbre1 : Entier ; Nbre2 : Entier) ;

Var Somme : Entier // **variable locale**

Début

 Somme= Nbre1+Nbre2 ;

 Ecrire (" la somme est", Somme) ;

Fin

Var N1, N2 : Entier ; // **variable globale**

Début

 Ecrire ("Entrez un premier entier") ;

 Lire(N1) ;

 Ecrire ("Entrez un deuxième entier") ;

 Lire(N2) ;

 Somme (N1 ; N2) ;

FinAlgo

IV. Mode de passage de paramètres

Il existe deux types de paramètres : **paramètre effectif** et **paramètre formel**

Un paramètre effectif est une valeur réelle reçue par le sous algorithme au cours de l'exécution de l'algorithme.

Exemple : Somme (N1 ; N2) ; N1 et N2 sont des paramètres effectifs.

Un paramètre formel est la définition du nombre et du type de valeurs que devra recevoir le sous algorithme pour s'exécuter correctement. On déclare les paramètres formels pendant la déclaration du sous algorithme.

Exemple :

Procédure *Somme* (Nbre1 : Entier ; Nbre2 : Entier) ;

 Nbre1 et Nbre2 sont des paramètres formels

Il existe deux modes de passage de paramètres : par valeur et par référence (adresse ou variable)

1. **Passage de paramètres par valeur** : il y'a copie de la valeur des paramètres effectifs dans les variables locales issues des paramètres formels du sous algorithme appelé. Le contenu des paramètres effectifs n'est pas modifié par les instructions du sous algorithme car on travaille avec une copie de la variable.

Exemple :

Soit l’algorithme suivant :

Algorithme Param_valeur

Procédure Val_Abs (n1 : Entier)

Début
 Si (n1<0) **Alors**
 n1=-n1 ;
 Finsi
 Ecrire (n1) ;
Fin

Var nbre : Entier ;

Début
 Lire (nbre) ;
 Val_Abs(nbre) ;
 Ecrire (nbre) ;

FinAlgo

Lors de l’exécution de cet algorithme avec la valeur -15, on a :

nbre	n1	Test	Ecran	
-15	Non déclaré			Avant l’appel de la procédure, on n’a que la variable nbre qui existe
-15	-15			A l’appel de la procédure, n1 est déclaré et reçoit une copie de nbre
-15	-15	vrai		On vérifie la condition
-15	15			
-15	15		15	
-15			-15	Retour à l’algo, il ne reste que la variable nbre et sa valeur initiale

2. **Passage de paramètres par référence** : ici on utilise l’emplacement mémoire de la variable c'est-à-dire que le paramètre formel se substitue au paramètre effectif durant le temps d’exécution du sous programme et lui transmet sa nouvelle valeur a la sortie. On se sert du mot clé **var**.

Exemple :

Soit l’algorithme suivant :

Algorithme Param_ref

Procédure Val_Abs (**Var** n1 : Entier)

Début
 Si (n1<0) **Alors**
 n1=-n1 ;
 Finsi
 Ecrire (n1) ;
Fin
Var nbre : Entier ;

Début
Lire (nbre) ;
Val_Abs(nbre) ;
Ecrire (nbre) ;
FinAlgo

Lors de l'exécution de cet algorithme avec la valeur -15, on a :

nbre	N1	Test	Ecran	
-15	Non déclaré			Avant l'appel de la procédure, on n'a que la variable nbre qui existe
	-15			A l'appel de la procédure, n1 se substitue à la variable nbre
	-15	vrai		On vérifie la condition
	15			
	15		15	
15			15	Retour à l'algo, il ne reste que la variable nbre et sa valeur a changé

V. Exécution d'un algorithme intégrant un sous algorithme

Algorithme Exe_Moyenne
Fonction Moyenne (Var x1, x2 : Entier) : Réel ;
 Var Moy : Réel ;
 Début
 Moy=(x1+x2)/2 ;
 Retourner Moy ;
 Fin
Var result : Réel
 n1, n2 : Entier
Début
 1. Ecrire ("Entrez le premier nombre") ;
 2. Lire (n1) ;
 3. Ecrire ("Entrez le deuxième nombre") ;
 4. Lire (n2) ;
 5. result= Moyenne (n1, n2) ;
 6. Ecrire (" la moyenne des deux nombres est ", result) ;
FinAlgo

	n1	n2	result	x1	x2	Moy	Ecran
2	14						
4	14	25					
5	14	25	19.5	14	25	19.5	
6							19.5

CONCLUSION :

Les fonctions et les procédures sont des éléments qui permettent d'améliorer la qualité des algorithmes. Cependant il faut veiller à les utiliser correctement afin de profiter de tous les avantages qu'elles offrent.

ACTIVITES DE CONSOLIDATION

1. Donner la syntaxe de déclaration d'une fonction et celle d'une procédure.
2. Ecrire une fonction qui calcule le PGDC de deux nombres entiers.
 - a. Ecrire un algorithme qui va utiliser cette fonction
 - b. Faire une exécution de cet algorithme avec les valeurs 15 et 9.
3. Ecrire une fonction qui permet de calculer la $n^{\text{ième}}$ valeur d'une suite de la forme $U_n = aU_{n-1} + b$, $U_0 = c$ (c, a, b sont des entiers naturels passés en paramètres)
4. Ecrire une procédure qui permet de lire 20 nombres réels et les stockent dans un tableau.
5. On désire produire une calculatrice qui va effectuer les tâches suivantes : addition, soustraction, multiplication, division, racine carrée, valeur absolue, le calcul de la puissance d'un nombre.
 - a. Comment peut-on procéder pour obtenir un algorithme simple ?
 - b. De combien de sous programmes a-t-on besoin ?
 - c. Ecrire les codes correspondant à chacun des sous programmes (fonctions et/ou procédures)
 - d. Ecrire l'algorithme qui va utiliser ces sous programmes.

NB : on va effectuer un passage de paramètres par référence pour le sous algorithme qui calcule la puissance d'un nombre et pour les autres algorithmes on va utiliser le passage de paramètres par valeur